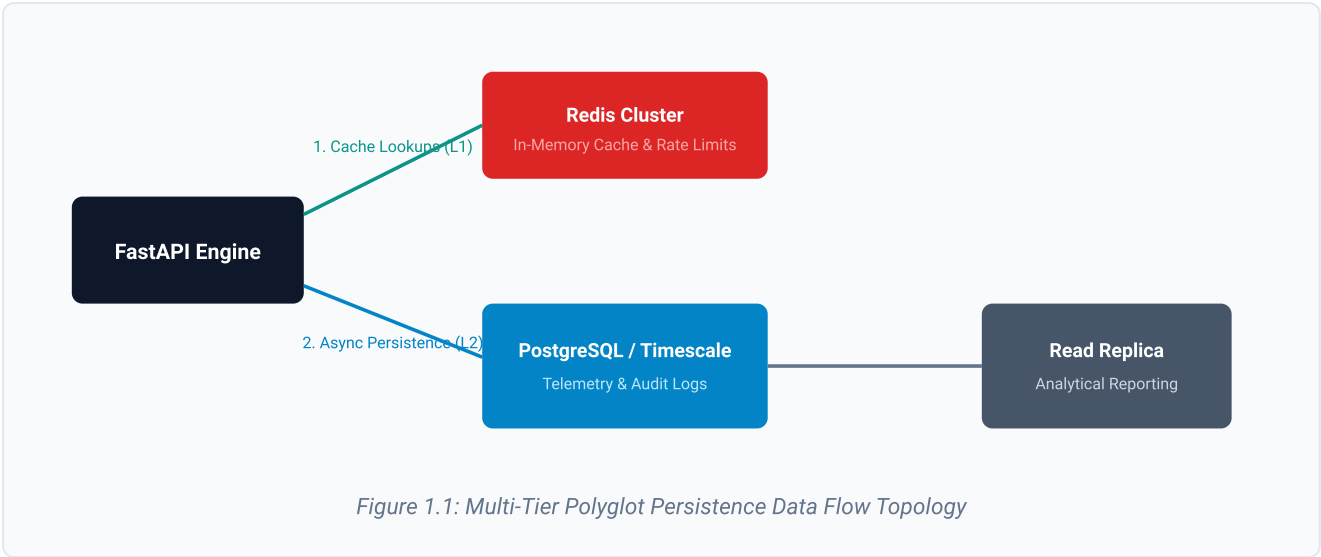


AI Weather API: Multi-Tier Storage Architecture

The **AI Weather API** backend utilizes a hybrid persistence model combining a relational database (PostgreSQL / TimescaleDB) for transactional records and time-series telemetry with an in-memory caching engine (Redis) for response acceleration and rate-limiting enforcement.



Storage Tier Objectives

Sub-Millisecond Read Speed Serves cached weather alerts and generated AI summaries directly from Redis memory to achieve <5ms key-value lookups.	Immutable Time-Series Tracking Persists incoming raw weather metrics in TimescaleDB hyper-tables for long-term climate analytics and model tuning.
--	--

Storage Design Principle

AI prompts and dynamic responses are cached alongside raw weather metrics to eliminate redundant LLM API invocations for queries occurring within a 5-minute window.

Database Storage & Engine Comparison Matrix

The system segregates data into specialized engine layers according to workload characteristics, write frequencies, and durability needs.

Data Domain	Engine Selected	Storage Type	Retention Strategy
Response Cache	Redis 7.x	In-Memory Key-Value	TTL Expiration (300 seconds)
Weather Metrics	TimescaleDB / Postgres	Relational Hyper-table	Partitioned Monthly (Indefinite)
AI Synthesis Logs	PostgreSQL (JSONB)	Document / Relational	90-Day Sliding Retention Window
User API Keys	PostgreSQL	ACID Relational	Permanent (Encrypted)
Rate Limit Buckets	Redis	Sliding Window Sets	TTL Expiration (60 seconds)

Key Architectural Trade-Offs

Eventual Consistency

Cache writes occur synchronously while database audit logging is executed via asynchronous background tasks to prevent blocking API responses.

Connection Pooling

Utilizes PgBouncer to manage database connections across high-concurrency serverless workers, preventing connection saturation.

Database Metric Target

Configured connection pool handles up to 5,000 active concurrent frontend requests while maintaining under 50 open server connections to PostgreSQL.

Relational Database Schema (DDL)

The SQL schema definition below establishes the relational tables required for API user management, raw metric logging, and AI inference auditing.

```
-- Core API User Account Management
CREATE TABLE api_clients (
  client_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_name VARCHAR(100) NOT NULL,
  api_key_hash VARCHAR(64) UNIQUE NOT NULL,
  rate_limit_rpm INT DEFAULT 60,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Time-Series Telemetry Table
CREATE TABLE weather_telemetry (
  record_id BIGSERIAL,
  city_name VARCHAR(100) NOT NULL,
  temperature_celsius NUMERIC(4,2) NOT NULL,
  humidity INT NOT NULL,
  wind_speed_ms NUMERIC(4,2) NOT NULL,
  weather_condition VARCHAR(50) NOT NULL,
  recorded_at TIMESTAMP WITH TIME ZONE NOT NULL,
  PRIMARY KEY (record_id, recorded_at)
);

-- Convertible to TimescaleDB Hypertable
SELECT create_hypertable('weather_telemetry', 'recorded_at', if_not_exists => TRUE);

-- AI Generation Log Audit Table
CREATE TABLE ai_inference_logs (
  inference_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  city_name VARCHAR(100) NOT NULL,
  prompt_tokens INT NOT NULL,
  completion_tokens INT NOT NULL,
  ai_response_text TEXT NOT NULL,
  execution_time_ms INT NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Database Indexing & Query Optimization

Index strategies are tailored for high-frequency city lookups, rapid time-series filtering, and audit log analysis.

```
-- Composite Index for Fast Location Telemetry Lookups
CREATE INDEX idx_telemetry_city_time
ON weather_telemetry (city_name, recorded_at DESC);

-- Partial Index for API Client Key Validation
CREATE INDEX idx_active_api_keys
ON api_clients (api_key_hash);

-- GIN Index for JSONB Metadata Querying on Logs
CREATE INDEX idx_ai_logs_city
ON ai_inference_logs (city_name, created_at DESC);
```

Query Execution Plan (EXPLAIN ANALYZE Target)

```
-- Fetching Latest Weather Record for a City
EXPLAIN ANALYZE
SELECT temperature_celsius, humidity, weather_condition
FROM weather_telemetry
WHERE city_name = 'London'
ORDER BY recorded_at DESC
LIMIT 1;

-- Expected Output: Index Scan using idx_telemetry_city_time (Cost < 0.05ms)
```

Index Scans over Seq Scans

Ensures all production queries hit dedicated B-Tree or Hypertable indexes, eliminating full sequential table scans.

Automated Vacuuming

Configures PostgreSQL autovacuum parameters aggressively to clean up dead tuples from frequent time-series writes.

Redis Cache Architecture & Key Layout

Redis acts as the frontline storage layer, storing active responses and tracking sliding-window rate limit counters to protect downstream services.

Redis Key Naming Schema

Pattern / Namespace	Type	TTL	Example Value
weather:cache:{city_slug}	String (JSON)	300 sec	{"temp": 18.5, "summary": "Cool..."}
ratelimit:{client_id}:{minute}	String (Integer)	60 sec	42 (Current request count)
ai:summary:lock:{city_slug}	String (Mutex)	10 sec	"1" (Prevents cache stampede)

Redis Python Caching Strategy

```
import json
import redis

redis_client = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)

def get_or_set_weather_cache(city: str, fetch_callback) -> dict:
    cache_key = f"weather:cache:{city.lower().strip()}"

    # 1. Attempt Read
    cached_data = redis_client.get(cache_key)
    if cached_data:
        return json.loads(cached_data)

    # 2. Cache Miss -> Execute Callback
    fresh_data = fetch_callback(city)

    # 3. Store in Cache with 5-Minute TTL
    redis_client.setex(cache_key, 300, json.dumps(fresh_data))
    return fresh_data
```

Database Connection Pooling & PgBouncer Config

Serverless workers create and destroy transient connections. To protect PostgreSQL from process exhaustion, PgBouncer is deployed in Transaction Pooling mode.

```
[databases]
ai_weather_db = host=127.0.0.1 port=5432 dbname=ai_weather_production

[pgbouncer]
listen_addr = *
listen_port = 6432
auth_type = md5
auth_file = /etc/pgbouncer/userlist.txt

; Pool Configuration
pool_mode = transaction
max_client_conn = 5000
default_pool_size = 40
min_pool_size = 10
reserve_pool = 10
reserve_pool_timeout = 5
```

Transaction Pooling

Assigns database connections only for the duration of an active transaction block, supporting high serverless worker concurrency.

Connection Scaling

Allows up to 5,000 incoming client connections to share a pool of 40 persistent PostgreSQL backend connections.

Production Best Practice

Prepared statements must be disabled or managed carefully when using PgBouncer in transaction pooling mode.

Database Replication & High Availability

High availability is achieved through Primary-Replica streaming replication combined with automated failover via Patroni / AWS Aurora cluster nodes.

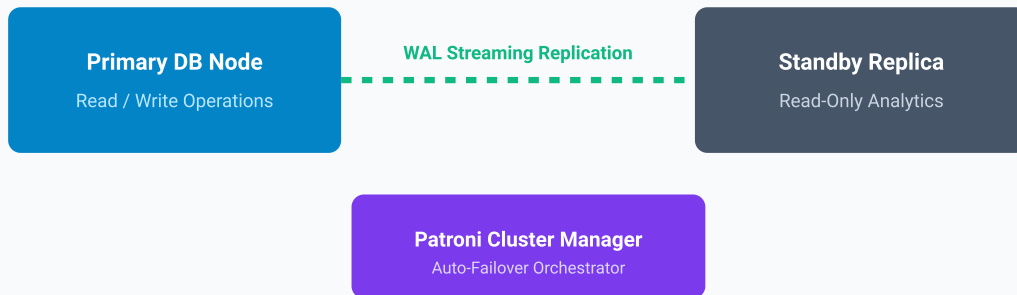


Figure 7.1: PostgreSQL Streaming Replication Topology

Replication Parameters (postgresql.conf)

```
# Primary Node Configuration
wal_level = replica
max_wal_senders = 10
wal_keep_size = 2048MB
synchronous_commit = off # Asynchronous replication for write speed
```

Database Security, Encryption & RBAC

All stored data is protected using AES-256 storage-at-rest encryption, TLS 1.3 transport encryption, and granular Role-Based Access Controls (RBAC).

```
-- Role Definition: Application API Worker
CREATE ROLE weather_app_user WITH LOGIN PASSWORD 'secure_password_here';

-- Grant Minimal Table Permissions
GRANT SELECT, INSERT, UPDATE ON weather_telemetry TO weather_app_user;
GRANT SELECT, INSERT ON ai_inference_logs TO weather_app_user;
GRANT SELECT ON api_clients TO weather_app_user;

-- Role Definition: Read-Only Analytics Role
CREATE ROLE analytics_reader WITH LOGIN PASSWORD 'analytics_pass';
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analytics_reader;
```

Security Compliance Checklist

Security Vector	Implementation Mechanism	Compliance Standard
Data at Rest	AWS KMS Managed Keys / LUKS Volume Encryption	SOC 2 / ISO 27001
Data in Transit	Enforced TLS 1.3 connections for DB & Redis	HIPAA / PCI-DSS compliant
Credential Storage	API Key hashes saved via SHA-256 with Salt	OWASP Top 10 Standard

Backup, Recovery & Disaster Recovery Plan

The database recovery strategy employs Point-In-Time Recovery (PITR) using Continuous Write-Ahead Log (WAL) archiving to S3 object storage.

Recovery Point Objective (RPO)

< 1 Minute: WAL logs are streamed near real-time to encrypted AWS S3 buckets.

Recovery Time Objective (RTO)

< 15 Minutes: Automated failover promotes standby replica in the event of primary failure.

Automated Backup Pipeline (pgBackRest)

```
# Continuous WAL Archiving Command
archive_command = 'pgbackrest --stanza=ai_weather archive-push %p'

# Automated Nightly Full Backup Cron Job
0 2 * * * pgbackrest --stanza=ai_weather --type=full backup
```

Disaster Recovery Drills

Automated restoration tests are executed weekly in isolated staging environments to verify WAL chain integrity.

Database Observability & Performance Metrics

Database health and performance metrics are monitored continuously using Prometheus PostgreSQL Exporter and custom Redis alerts.

Critical Database Metrics Target

Metric Name	Healthy Range	Critical Alert Trigger
Buffer Cache Hit Ratio	> 99.0%	< 95.0% for 10 mins
Active DB Connections	10 - 35 connections	> 45 connections (Max 50)
Replication Lag	< 100 milliseconds	> 5,000 milliseconds
Redis Memory Usage	< 60% max memory	> 85% (Eviction trigger)

Operational Summary

The multi-tier persistence model provides low latency, strong data durability, and automated recovery, creating a robust foundation for the AI Weather API.